Bachelor Degree Project

A Comparative Performance Study of REST and GraphQL APIs:

Latency and Throughput under Varying Query Complexity



Author: Ranim Hanna Malko
Supervisor: Jenny Lundberg
Semester: VT 2026
Subject: Computer Science

Abstract

Modern web and distributed systems rely heavily on Application Programming Interfaces (APIs), with Representational State Transfer (REST) and GraphQL serving as the primary architectural styles. While performance metrics such as latency and throughput are crucial for user experience and system efficiency, existing research often lacks a detailed analysis of how these APIs perform under varying levels of query complexity. This thesis investigates the performance differences between REST and GraphQL APIs by examining their behavior under comparable workloads and evaluating the impact of increasing query complexity on scalability. Using a design science methodology, a controlled experimental environment was developed with Python, Flask, and SQLite, while performance was evaluated using the Locust load testing tool. Measurements were conducted across three levels of query complexity, ranging from simple lookups to deeply nested relational data, under workloads of 50, 200, and 500 concurrent users. The results indicate that REST consistently delivers lower latency and higher throughput across all test scenarios, demonstrating superior scalability under high concurrency. While both architectures perform similarly for simple queries, GraphQL performance degrades more substantially as query complexity increases due to the additional server-side overhead associated with query parsing and resolver execution. However, GraphQL offers advantages when retrieving deeply nested relational data, where equivalent functionality in REST may require multiple client requests. The findings suggest that REST is better suited for high-volume, performance-critical systems, whereas GraphQL is more appropriate for applications requiring flexible and efficient retrieval of complex, interconnected data.

Keywords: REST APIs, GraphQL, API Performance, Latency, Throughput, Query Complexity

Contents

1 Introduction---- 3
1.1 Background----4
1.2 Related work---- 5
1.3 Problem formulation---- 6
1.4 Motivation---- 7
1.5 Results----7
1.6 Scope/Limitation----7
1.7 Target group---- 8
1.8 Outline---- 8
2 Method----9
2.1 Literature Review----9
2.2 Design Science---- 10
2.3 Reliability and Validity---- 10
2.4 Ethical Considerations---- 11
3 Theoretical Background---- 12
3.1 Application Programming Interfaces (APIs)---- 12
3.2 Latency, Throughput, and Query Complexity---- 12
3.3 REST as an Architectural Style---- 13
3.4 GraphQL as a Query Language and Runtime---- 14
3.5 Architectural Differences of REST and GraphQL---- 16
4 Implementation----17
4.1 Technology Stack---- 17
4.2 Query Complexity & Workload Design---- 18
5 Results----20
6 Analysis and Discussion---- 24
6.1 REST vs GraphQL (Latency & Throughput)----24
6.2 Query Complexity Impact---- 24
6.3 When to use REST vs GraphQL---- 25
7 Conclusions and Future Work---- 27
References---- 28
A Appendix---- 31
AI Statement----31
Appendix 1:----32
Appendix 2:----35
Appendix 3:----37
Appendix 4:----39
Appendix 5:----45
Appendix 6:----45
Appendix 7:----45

1 Introduction

In software engineering and the development of distributed systems, Application Programming Interfaces (APIs) serve as fundamental interfaces that enable communication and interaction between different software components. This bachelor's degree project in Computer Science explores the evolving landscape of web API architectures, which are essential for the operation of digital services such as e-commerce and social media platforms. For more than two decades, Representational State Transfer (REST) has remained the dominant architectural style, utilizing a stateless, resource-oriented design where data is exposed through multiple endpoints. However, GraphQL, introduced by Facebook as an alternative, has gained prominence by offering a flexible query language and runtime that allow clients to request exactly the data they need through a single endpoint.

API performance, particularly latency and throughput, is a critical factor in determining overall service quality and user experience. While existing research has extensively compared REST and GraphQL under general workloads, a knowledge gap remains regarding how query complexity, particularly under concurrent workload conditions, affects these performance metrics. REST is often limited by fixed response structures, which can lead to over-fetching or under-fetching of data, while GraphQL provides greater flexibility for handling complex queries, but this can introduce higher server-side overhead due to query parsing and resolver execution.

The purpose of this thesis is to address this gap through a controlled experimental study investigating how varying levels of query complexity affect the latency and throughput of REST and GraphQL APIs. By providing an empirical evaluation of these two architectures, the research aims to support software developers and system architects in making informed design decisions that optimize performance and resource efficiency. Understanding these trade-offs is important for ensuring scalability in performance-critical applications.

1.1 Background

REST has become the industry standard for Web APIs over the past two decades due to its simplicity, flexibility, and scalability in backend systems and server-side web services [5]. In RESTful systems, data is exposed through multiple distinct endpoints, each identified by a Uniform Resource Identifier (URI) [1, 2, 11]. REST follows a stateless client–server architecture, where each request is processed independently without relying on prior interactions. It typically uses standard HTTP methods, such as GET, POST, PUT, PATCH, and DELETE, to perform create, retrieve, update, and delete operations on resources [5, 6]. The stateless nature of REST requires that each client request contain all the necessary information for the server to process it correctly [1, 15].

GraphQL is an open-source data query language and server-side runtime designed for executing queries against a strongly typed schema [2, 28]. Originally developed by Facebook in 2012 and released as open source in 2015, GraphQL provides an alternative approach to

data fetching [2, 4, 16]. Unlike REST, which uses multiple endpoints, GraphQL usually uses a single endpoint to handle all requests. It is based on a strongly typed schema and represents data as a graph. This allows clients to request exactly the fields they need in one query, making data fetching more flexible and efficient [1, 5, 16, 23]. Query complexity refers to the computational and resource cost required to execute a data request, ranging from simple resource retrieval to complex queries involving deeply nested relationships [24, 30].

In modern distributed and web systems, latency and throughput are important factors in overall quality of service. Latency refers to the time it takes for a service to respond to a request, while throughput describes the number of requests a system can handle within a given period [1]. Efficient backend operations are necessary to support complex real-time updates and large-scale data processing [10]. Performance also has a direct effect on user experience, as slow response times can lead to user dissatisfaction and reduced engagement [1].

1.2 Related work

Several studies have investigated the performance of REST and GraphQL using different evaluation techniques, including load testing, stress testing, empirical system measurements, and simulated API workloads [1, 3, 4, 8, 9, 10, 11, 12, 29]. Common tools such as Apache JMeter, k6, and custom node.js based test environments are used to evaluate metrics including response time, throughput, CPU and memory usage, and payload size under varying concurrency levels.

Several previous studies focus on system efficiency under load using real-world or simulated environments. REST is often reported to achieve higher throughput and lower response times under high concurrency due to its stateless design and caching mechanisms [1, 4, 9, 10, 11]. In contrast, GraphQL typically demonstrates lower resource consumption in terms of CPU and memory but may experience higher latency under heavy load conditions [1, 10, 11]. Load testing studies further indicate that REST performs more effectively under high concurrent traffic, while GraphQL tends to exhibit more stable resource utilization patterns [10, 11].

Other studies emphasize data transfer efficiency using API-level testing and request-based evaluations. GraphQL consistently reduces over-fetching by allowing clients to specify required fields, resulting in lower payload sizes compared to REST [3, 4, 8, 12]. REST, however, often produces more predictable performance in simple retrieval operations due to its fixed endpoint structure, although this may increase unnecessary data transfer [4, 8, 11].

In addition, several works investigate performance in complex query or multi-entity scenarios, often using nested queries or aggregated API requests as test cases [9, 29]. These studies show that GraphQL is more efficient in handling complex and interconnected data by reducing the number of required requests, while REST performs better in simpler single-resource operations with lower structural overhead [9, 11, 12]. Enterprise-level evaluations also

highlight GraphQL advantages in reducing payload size and improving response efficiency in nested query environments [29].

1.3 Problem formulation

Existing research primarily evaluates REST and GraphQL using controlled load testing and system-level performance metrics, providing valuable insights into throughput, latency, and resource usage. However, most studies examine workload and query structure independently, with limited focus on how query complexity interacts with increasing concurrency. A smaller body of research considers complex, nested, or multi-entity queries, but does not systematically vary these query structures alongside different workload levels. Consequently, it remains unclear whether the relative performance advantage of REST and GraphQL depends on the combination of workload and query complexity, rather than either factor alone. This study addresses this gap by adopting a comparable experimental load-testing approach that systematically varies both query complexity and workload.

RQ1: How do REST and GraphQL APIs differ in latency and throughput under comparable workloads?

To answer this question, both API styles are implemented using a shared backend and identical datasets to ensure a fair comparison. Controlled workloads are generated using the load testing tool Locust, and key performance metrics (latency and throughput) are measured and compared across both implementations, then evaluated in relation to findings from existing literature.

RQ2: How does query complexity affect latency and throughput in REST and GraphQL APIs as concurrent workload increases?

This question evaluates the impact of query complexity on API performance across three complexity levels, ranging from simple resource requests to complex queries involving nested and related data, while increasing the concurrent workload at each level. This allows the analysis to assess whether one architecture demonstrates better scalability as both query complexity and workload intensity increase.

RQ3: Under what conditions is REST or GraphQL more suitable based on performance and query complexity?

Building on the results of RQ1 and RQ2, this question integrates the experimental findings with the literature to identify key trade-offs, particularly regarding how query complexity and performance requirements affect the suitability of REST or GraphQL. These insights ultimately guide the selection of an appropriate API architecture.

1.4 Motivation

Beyond the identified knowledge gap, this study is motivated by the technical and practical importance of API performance. Response time and reliability directly affect user experience, while efficient and responsive APIs are essential for delivering high-quality digital services, such as e-commerce platforms and social networks. Furthermore, aligning the API architecture with expected query complexity and traffic patterns can help reduce unnecessary resource consumption. The findings of this study are intended to give developers and architects an empirical basis for that choice, complementing the general-purpose guidance already available in the literature.

1.5 Results

The results are presented as a comparative analysis showing how both architectures behave as concurrent users load and query complexity increase. The experimental results indicate that REST and GraphQL perform comparably under low and medium query complexity levels, as well as under low and medium concurrency workloads. However, when aggregating latency and throughput across all query complexity levels at 500 concurrent users, REST demonstrates a performance advantage over GraphQL of 6.5-8%.

Query complexity has an increasing impact under high workload conditions, leading to a wider performance gap between the two architectures. Under these conditions, with 500 concurrent users executing complex queries, REST outperforms GraphQL by 14.3-15% in both latency and throughput. REST demonstrates better scalability when concurrency and query complexity increase simultaneously. In contrast, GraphQL provides greater flexibility in data retrieval, particularly in scenarios where REST would require multiple API calls to obtain related data, potentially increasing overall latency. The findings from RQ1 and RQ2 support RQ3 by demonstrating that REST is more suitable for performance-critical systems, whereas GraphQL is better suited for applications requiring flexible and complex data retrieval.

1.6 Scope/Limitation

This thesis compares REST and GraphQL within a controlled experimental environment using Python, Flask, Graphene, SQLite, and Locust. The evaluation is limited to key performance metrics, specifically latency (response time) and throughput (requests per second), under three query complexity levels and three workload conditions. The study does not consider advanced deployment platforms, such as Docker, nor does it address microservices architectures, authentication, or authorization mechanisms. Furthermore, the study does not incorporate advanced database management systems, such as MySQL or MongoDB, instead relying on SQLite to maintain consistency within the controlled

experimental setup. System-level performance metrics, such as CPU and memory utilization, are only observed at a basic level and are not included as part of the systematic analysis.

1.7 Target group

The primary target audience for this research includes software developers who implement REST or GraphQL APIs within applications and systems, as well as software architects responsible for making design decisions related to API selection, and backend engineers concerned with optimizing system performance. Secondary beneficiaries include researchers in software engineering and distributed systems, particularly those interested in empirical comparisons of API architectures and methods for performance evaluation.

1.8 Outline

Chapter 1, *Introduction*, establishes the research problem, motivation, and the specific research questions. Chapter 2, *Method*, describes the literature review to build a theoretical foundation and a design science approach to conduct a controlled experiment using the Locust tool. Chapter 3, *Theoretical Background*, provides essential technical context detailing the architectural principles of REST and GraphQL, and the definitions of latency, throughput, and query complexity. Chapter 4, *Implementation*, explains the Python/Flask-based backend and the design of the workloads and query-complexity levels used for comparison. Chapter 5, *Results*, presents the empirical data and performance trends of both architectures across the tested workloads. Chapter 6, *Analysis and Discussion*, interprets these findings against the literature and answers the research questions. Chapter 7, *Conclusions and Future Work*, summarizes the study's contributions, provides final recommendations on API suitability, and suggests directions for future research.

2 Method

This study combines a literature review with a design science methodology to address the research questions. The literature review establishes the theoretical foundation by synthesizing existing research on REST and GraphQL performance, while the design science component develops and evaluates an experimental artefact consisting of two functionally equivalent API implementations and a controlled test environment. Together, these components generate empirical evidence to address the gap identified in the literature.

The methodology proceeds in three stages:

1. Literature review: establishes the current knowledge on REST and GraphQL performance, including latency, throughput, and the impact of query complexity.
2. Design science: functionally equivalent REST and GraphQL APIs are implemented and evaluated with Locust across three workload levels and three query-complexity levels.
3. Comparison and validation: the experimental results are first examined for internal consistency and then compared against the literature to answer RQ1–RQ3.

This combined methodology was selected because neither approach alone is sufficient to answer the research questions. The literature review cannot fully explain the interaction between query complexity and workload, as previous studies typically examine these factors independently. A controlled experiment alone without grounding in prior research would provide limited external validity. By combining both approaches, the experimental findings can be interpreted and validated within the context of the existing body of knowledge.

2.1 Literature Review

A literature review is conducted to collect, analyze, and evaluate existing research on REST and GraphQL performance. Only relevant studies and theoretical contributions are included, and the selected sources are critically assessed to ensure accuracy, relevance, and completeness. The review focuses on performance metrics, comparative analyses, and guidelines for selecting appropriate architectural styles in different application scenarios. The purpose of the literature review is to establish a theoretical foundation for the study and to identify previously observed trends and findings. It also supports experimental components by highlighting commonly used performance metrics, evaluation methods, and known differences between REST and GraphQL. By integrating insights from prior research with empirical results, the study aims to provide a more reliable and comprehensive analysis.

Most selected sources are peer-reviewed research articles and technical reports addressing latency and throughput, resource utilization (CPU and memory), query efficiency, and system behavior under varying workloads. Google Scholar was used as the primary search engine, providing access to books, theses, journal articles, and other academic publications from publishers, professional societies, universities, and research institutions [26]. The search

process was supplemented by the Association for Computing Machinery (ACM) Digital Library and the Institute of Electrical and Electronics Engineers (IEEE) Xplore Digital Library, which provide access to technical content publications from IEEE and its partners [25]. Additionally, official documentation for Python, Flask, REST, and other related frameworks and platforms was consulted to support the technical aspects of the study. The following search terms were applied:

- REST API vs GraphQL performance
- REST API vs GraphQL latency and throughput
- REST API vs GraphQL effects of query complexity
- REST API query complexity and GraphQL query complexity
- API query complexity

2.2 Design Science

This study follows a design science methodology, used for designing and validating solutions to practical problems [13, p.9], and for evaluating IT artefacts to improve understanding of a problem context [14, p.77]. Within design science, several evaluation methods exist for assessing an IT artefact. This study uses a controlled experiment, which enables direct, repeatable measurement of the theoretical concepts identified in the literature review and allows weaknesses in those assumptions to be identified empirically [14, p.80].

The artefacts developed in this study consist of a REST API implementation, a functionally equivalent GraphQL API implementation, and a controlled Locust-based load testing environment that evaluates both APIs under identical workload and query-complexity conditions. Together, these artefacts address RQ1 and RQ2 by enabling a direct comparison of latency and throughput measurements, while supporting RQ3 by providing empirical evidence for identifying the conditions under which each API style performs better. Both APIs are implemented in Python and Flask, sharing the same runtime, Object-Relational Mapper (ORM), and database layer, so that query complexity and workload are the only systematically varied factors between comparable test scenarios. The experimental results are analyzed and compared with the literature to assess performance under different conditions and workload levels.

2.3 Reliability and Validity

To ensure the quality of the research, several measures are taken to improve reliability. The same test environment is used for all test scenarios. Both APIs are implemented using the same technology stack, and identical workloads are applied to both REST and GraphQL. In addition, the experiments are repeated multiple times to allow clear and consistent conclusions to be drawn. However, the experiment is conducted in a simplified environment, while real-world systems may include additional factors such as network latency and other

performance-related conditions that could affect the results and lead to different conclusions. Potential threats to validity also include the simplified design of the APIs compared to real-world systems, as well as limitations in the workloads used. To reduce these threats, the experimental results are compared with findings from existing literature, and multiple test scenarios are conducted to improve reliability.

2.4 Ethical Considerations

This study does not involve human participants, surveys, or interviews. Ethical considerations related to sensitive personal data are therefore minimal. No personal data is collected or processed, and GDPR is not applicable. All sources used in the literature review are properly cited. The experimental setup is conducted in a local environment, ensuring that no external systems are affected, and both REST and GraphQL are evaluated fairly under the same experimental conditions.

3 Theoretical Background

3.1 Application Programming Interfaces (APIs)

An API is a set of rules, tools, and protocols that allow various systems, whether web-based or desktop applications, to interact and exchange information with servers without requiring additional backend development, which substantially simplifies application development and integration [10, 12]. In a client–server architecture, APIs act as a bridge between the user (client) and the data management system (server) [10, 22], supporting an architectural style in which the user interface on the client side is separated from the business logic and data storage on the server side [3, 10]. This separation ensures that the server does not need to be aware of the client's user interface, while the client does not need to understand the complexity of the underlying business logic [31].

Communication between client and server is typically established using the Hypertext Transfer Protocol (HTTP). The client sends a request to an endpoint, which is a specific URL representing a resource (data source), and the server processes this request and returns a representation of the data, often in JSON or XML format [1, 22, 27]. APIs ensure that interactions are carried out correctly by defining how data is sent, received, and processed, thereby enabling different systems to interpret each other’s data structures [12]. APIs have become the backbone of modern software development and are widely used in cloud computing and microservices architectures. In modern distributed systems, applications are often divided into smaller services that communicate through APIs. This loose coupling allows different teams to develop, deploy, and scale individual components of a system independently [9, 22]. Several Web API architectural styles have been developed in recent years, each representing a set of principles intended to improve system properties to meet the needs of developers and applications across various scenarios [10]. Common styles include REST, SOAP (Simple Object Access Protocol), GraphQL, and gRPC (Remote Procedure Call).

3.2 Latency, Throughput, and Query Complexity

When evaluating API performance, key considerations include latency, throughput, and resource utilization (such as CPU and memory usage), as these factors determine whether a system meets Quality of Service (QoS) requirements [1, 22].

Latency (response time, measured in milliseconds) refers to the total time between a client sending a request and receiving a complete response from the server [1, 21], commonly measured as round-trip time (RTT), which includes both request and response transmission over the network [7]. Latency can be analyzed using average response time, which reflects overall user experience, and maximum response time, which is important for identifying worst-case performance and slow queries [1].

Throughput**,** measured as requests per second (RPS), quantifies the number of successful requests an API can process within a given time interval [1], reflecting the overall processing capacity and efficiency of the system [7, 21]. Scalability refers to a system's ability to handle increasing request volumes and larger datasets without degrading performance [9]. High scalability ensures that response times remain stable even under growing concurrent workloads [7, 21]. Network efficiency is closely related to data transfer behavior, particularly over-fetching and under-fetching, which are common performance challenges in REST-based systems tied directly to payload size and the number of network calls required to fulfill a request [7, 27, 31].

Query complexity in web APIs refers to the computational and resource cost required to execute a specific data request [24, 30]. It can be characterized by factors such as the number of resources requested, the depth of relationships between entities, filtering, searching and sorting operations, and the number of database operations required to retrieve the data, which substantially increase computational cost [7]. In REST APIs, complexity is often associated with the number of endpoint calls required to retrieve a complete dataset, GraphQL instead defines complexity based on query depth and the relationships within a single query structure [11, 30]. The impact of query complexity on performance varies with system design, but generally affects data retrieval, backend processing, and network usage. Simple queries typically result in low latency, while complex queries require more intensive backend processing and interaction with multiple data sources [21]. In REST, complex data requirements often lead to under-fetching, where multiple sequential requests are needed to obtain a complete dataset [11, 28, 31].

3.3 REST as an Architectural Style

REST is an architectural style built on the Hypertext Transfer Protocol (HTTP) [5]. Its core principles include client–server separation, stateless communication, cacheability, a uniform interface, and a layered system architecture [11]. Client–server separation decouples user interface (client) concerns from data management and business logic (server), enabling the independent evolution of components and improving portability across platforms [10, 11, 31]. Statelessness further supports scalability by requiring each request to include all the information necessary for the server to process it, without the server retaining client context between requests [1, 5, 9, 31].

REST also emphasizes cacheability to enhance efficiency. HTTP caching mechanisms allow responses to be stored at various points along the request–response path, such as at the client or an intermediary proxy, enabling the reuse of previously retrieved data without contacting the server, which reduces server load and latency for frequently accessed resources [1, 5, 9, 22, 31]. The uniform interface constraint standardizes how clients and servers interact through resource identification via Uniform Resource Identifiers (URIs) and the manipulation of resources through their representations [11, 31]. REST is inherently resource-oriented, where

entities such as users or posts are treated as resources accessed via specific endpoints and commonly represented in formats such as JSON or XML [2, 9, 1, 11, 31, 22, 3].

REST architectures are also layered, allowing intermediaries such as load balancers and gateways to sit between client and server without affecting communication. This abstraction improves scalability, security, and availability by hiding backend complexity from clients, who need not know whether they are connected directly to the end server [11, 31]. REST relies on standard HTTP methods to support CRUD operations on resources [1, 3]:

- GET: retrieves a representation of a resource without modifying it.
- POST: creates a new resource or submits data to the server.
- PUT: updates an existing resource by replacing it with the provided data.
- DELETE: removes a resource from the server.
- PATCH: applies partial modifications to a resource.

Advantages and Limitations of REST

REST has become widely adopted due to its simplicity, scalability, and strong integration with HTTP, making it easy to implement and enabling any HTTP-capable system to communicate with REST-compliant services [1, 5, 28]. Its stateless nature, in which each request carries all the information required for processing, facilitates scalability and makes REST particularly suitable for enterprise applications and large-scale distributed systems [31]. REST also leverages HTTP caching, which improves performance by reducing server load and response times for frequently accessed data. It also benefits from a large ecosystem of tools, libraries, and widespread developer familiarity, which supports its continued adoption [5, 9, 28].

REST also presents limitations. A common issue is **over-fetching,** where endpoints return more data than required because of fixed response structures [6]. Conversely, **under-fetching** can occur when a single endpoint does not provide sufficient information, requiring multiple sequential requests to gather related data [9, 11, 23, 31]. As systems scale, managing and versioning numerous endpoints can increase complexity and maintenance effort, and backend changes may require corresponding client-side updates, reducing development flexibility [1, 9, 22].

3.4 GraphQL as a Query Language and Runtime

GraphQL is a query language and server-side runtime that provides an alternative approach to fetching data [2, 4, 16]. It was introduced to address limitations of RESTful architectures, particularly those related to data-fetching efficiency [5, 10, 11, 12, 29]. As a server-side runtime, GraphQL is agnostic to the underlying database or storage system, allowing flexible integration with various data sources [1].

GraphQL architecture centers on schemas, types, queries, mutations, and resolvers. The **schema** is the foundation of a GraphQL API, defining the available data structures and relationships and acting as a contract between client and server, ensuring both understand the data structure [27, 28]. **Types** define the structure of the data fields and represent the fundamental units within the schema [1]. These can include primitive types (such as integers and strings), enumerations, and complex object types [31]. Clients retrieve data through queries, and a **query** specifies exactly the data required in response, enabling efficient data fetching [1, 5, 11, 27]. **Mutations** handle write operations such as creating, updating, and deleting data on the server [1, 27]. Each field within a schema is associated with a **resolver** function, which is responsible for retrieving the requested data from underlying data sources, such as databases or external APIs [27, 28, 29].

Unlike REST, which commonly relies on multiple resource-specific endpoints, GraphQL typically operates through a single endpoint (usually an HTTP POST request), where clients specify exactly which data is required [1, 28]. This eliminates common REST issues such as over-fetching and under-fetching by allowing related data to be retrieved in a single request [6, 9, 11, 12, 24, 27, 29, 31].

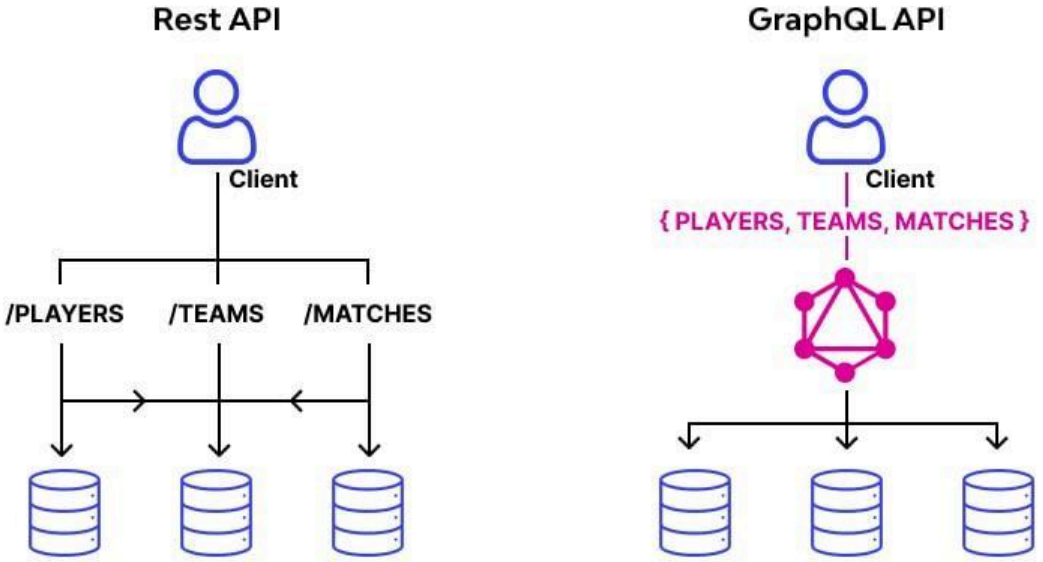
Advantages and Limitations of GraphQL

By enabling precise data retrieval, GraphQL reduces network payload size and minimizes the number of requests required, improving overall efficiency [9, 12, 23, 27, 31]. Its strongly typed schema also enables built-in validation, self-documentation, and API introspection, letting clients discover available types and fields [11, 24, 27]. It also supports flexible API evolution since fields can be deprecated without versioning the entire API [5, 27]. Some research also suggests GraphQL can offer improved CPU and memory efficiency for complex, high-volume queries through a single endpoint [1, 10, 22].

However, GraphQL also has limitations. Its adoption introduces additional complexity and a steeper learning curve, since developers must define schemas and implement resolver logic [4]. Since clients specify their data requirements while the server is responsible for resolving queries, GraphQL increases server-side complexity by requiring resolvers to retrieve and combine data across multiple sources and relationships [1, 21]. Additionally, GraphQL lacks native support for standard HTTP caching because each query can be unique, making caching mechanisms such as CDNs more difficult to apply effectively compared to REST [27, 31]. Complex or deeply nested queries may also introduce performance challenges, such as the N+1 query problem, which can increase server load and latency [5, 9, 29]. Furthermore, the flexibility of GraphQL queries can create security risks, including denial-of-service (DoS) attacks and injection vulnerabilities, if queries are not properly validated and controlled [21, 24, 27, 28].

3.5 Architectural Differences of REST and GraphQL

Aspect	**REST**	**GraphQL**
Endpoint Structure	resource-oriented, multiple endpoints	single endpoint
Data Fetching	server determines the data structure for each endpoint. (Fixed responses) can cause over-/under-fetching (N+1 problem)	client specifies exactly the fields needed, no over-/under-fetching
Flexibility & API evolution	new data requirements often need new endpoints or versioning (/v1, /v2)	clients change queries without server changes (no code modifications), no versioning required
Client–server responsibilities	stateless architecture, server defines the data structure, client follows it	client defines data needs, server resolves via resolvers, increasing server-side complexity
Caching	strong HTTP caching (browsers, CDNs, reverse proxies) via unique resource URLs	limited caching, queries can be unique, requiring custom resolver- or application-level caching [21, 29]
Design Philosophy	resource-oriented, stateless, HTTP-standards based	query language and runtime centered on a strongly typed, graph-based schema



(https://www.wallarm.com/what/graphql-vs-rest-all-that-you-must-know)
Figure 3.1:GraphQL vs REST API architecture styles differences.

4 Implementation

This experiment measures two performance metrics: latency and throughput. These metrics are also commonly used in prior studies, which measured them using tools such as Apache JMeter and k6 [4, 8, 9, 10, 11, 12]. This study uses Locust, a Python-based load testing tool, chosen because it integrates directly with the Python-based API stack described below, allowing test scenarios to be written in the same language as the APIs under evaluation. Locust records latency and throughput, enabling a quantitative comparison between REST and GraphQL [20]. To ensure a fair comparison, two functionally equivalent APIs, one REST and one GraphQL, are implemented using an identical backend and evaluated under the same conditions. Both implementations are provided in Appendix 1.

4.1 Technology Stack

Python was used to implement both APIs because it provides mature support for REST (via Flask) and GraphQL (via Graphene) within a single ecosystem, letting both APIs share the same runtime, ORM, and database layer. Python's high-level data structures and straightforward object-oriented syntax also suited rapid development within the thesis timeframe [17].

Flask was chosen over larger frameworks such as Django or FastAPI because it is a minimal, unopinionated microframework that exposes routing and request handling directly, without extra middleware or ORM abstractions that could add uncontrolled overhead to the measured request path [18]. This also made it simple to mount the GraphQL endpoint, via Graphene's Flask integration, alongside the REST endpoints in a structurally comparable way.

Graphene was used to implement the GraphQL schema and resolvers. It is a widely used, actively maintained Python GraphQL library that follows a code-first approach. The schema is defined directly in Python rather than a separate schema file and provides built-in integration with SQLAlchemy models [33]. Other Python GraphQL libraries exist, such as Ariadne and Strawberry.

SQLite was chosen as the database because it is serverless, requiring no separate database process, network connection, or configuration, which makes it lightweight and easy to integrate into a local testing environment. Its official documentation recommends SQLite primarily for single-application, local storage rather than high-concurrency client/server workloads [19], which suits this experiment, conducted entirely in a local environment and intended to evaluate application behavior rather than production-scale throughput on a client/server database such as PostgreSQL or MySQL.

SQLAlchemy is used as the ORM connecting both APIs to the database. It represents database tables and relationships as Python classes and manages the translation between Python objects and SQL queries, so both APIs interact with the same domain model rather than writing raw SQL [32]. The database schema has three tables: users, posts, and comments,

related through SQLAlchemy ORM relationships (Appendix 2). Each user has multiple posts, and each post has multiple comments. A seeding script populates the database with 500 users, 10 posts per user, and 10 comments per post, 55,500 records in total, simulating a realistic blog system with relational depth. This is somewhat larger than the 1,000–10,000 record datasets typically used in prior work [3], to better reflect nested-query behavior under workload.

4.2 Query Complexity & Workload Design

Both APIs expose three comparable levels of query complexity: a simple lookup of a single user, a medium query retrieving a user's posts, and a complex query retrieving a user's posts together with their comments. Table 4.1 summarizes the corresponding REST endpoint and GraphQL query for each level.

Query Complexity	**REST Endpoint (GET method)**	**GraphQL Query (POST method)**	**Description**
Simple	/user/<id>	{ user(id:1){ id name } }	Retrieves basic user information
Medium	/user/<id>/posts	{ user(id:1){ posts { title } } }	Retrieves all posts associated with a user
Complex	/user/<id>/full	{ user(id:1){ posts { comments { text } } } }	Retrieves user posts and their associated comments

Table 4.1: The query complexity levels, their descriptions, and the corresponding API.

Both implementations query the database in proportion to the data they return. REST achieves this naturally, as each complexity level has its own endpoint with a correspondingly scoped join. In GraphQL, this behavior had to be implemented explicitly. The resolver inspects the fields selected in the incoming query and joins only the required relationships, so that simple, medium, and complex GraphQL requests impose a database load comparable to their REST counterparts.

Several optimizations commonly used in REST and GraphQL applications were intentionally excluded to keep the comparison focused on request handling and database costs. Authentication, cookies, and caching were excluded from the experimental scope. In particular, REST fixed, resource-based URLs are well suited to standard HTTP and CDN caching, whereas GraphQL single endpoint with variable queries is generally less compatible with these mechanisms. DataLoader, a batching utility that combines multiple database calls to mitigate the N+1 query problem when resolving lists of parent objects, was also not used, since the schema in this study resolves only a single user per request rather than a list of users, leaving no opportunity for batching.

Prior studies have used a wide range of simulated workloads, from approximately 50 to 1,500 concurrent users [2, 3, 5, 11, 21]. A concurrent user represents one simulated client that repeatedly sends requests for the duration of the test, independently of other simulated users. 50 concurrent users means 50 separate request streams running continuously and in parallel for the whole test, so a higher count represents a heavier workload, closer to many real users accessing the system simultaneously. Based on these findings, this study adopts three levels of simulated load: 50 (low), 200 (medium), and 500 (high) concurrent users, following the same three-tier design used in [21]. These levels represent typical and moderate testing scenarios in which performance differences between REST and GraphQL are expected to become more apparent.

Each experiment was conducted for two minutes, with a five-second ramp-up period, allowing the system to reach a stable state while generating enough requests for reliable measurements. The ramp-up period is the time Locust takes to gradually start all simulated users at the beginning of a test rather than launching all of them simultaneously. Some previous studies measured a fixed number of requests, such as 100 per API [1, 3], because Locust does not natively support stopping after a fixed request count. A time-based approach was adopted instead, consistent with the methodology used in [10, 11].

To improve reliability, each test was repeated five times under identical conditions and averaged. The experimental procedure was applied to both APIs across all query complexity levels and for each workload level, with all query types defined in a single Locust script (Appendix 3) and unrelated tasks disabled during each run to isolate one query type at a time. In total, 90 experiments were conducted, 45 for workload variations and 45 for query complexity, with a total execution time of approximately 180 minutes.

Locust results were exported as CSV files and analyzed in Python using Pandas and Matplotlib (Appendix 4). The analysis compares latency and throughput across workload levels (RQ1) and across query complexity levels (RQ2), with the findings interpreted against existing literature to answer RQ3.

This study restricts its performance evaluation to latency and throughput and does not collect CPU or memory utilization data during test execution. Resource utilization has already been measured for REST and GraphQL under comparable conditions in prior work alongside latency and throughput [1, 10, 21]. Rather than duplicating this measurement, the present study treats those findings as established context and focuses on the specific gap identified in Section 1.3.

5 Results

The experimental results are analyzed to address research questions related to latency and throughput under varying workloads and query complexities. Reported values are averaged across five runs, the full dataset is presented in Appendix 5.

Latency Analysis: The values in Table 5.1 are averaged across all query complexity levels, alongside the minimum and maximum observed response times per condition. REST consistently maintained lower average latency than GraphQL across all workload levels, although the difference became more pronounced as the workload increased. At 500 concurrent users, REST achieved a lower average latency than GraphQL, representing an improvement of 8%. Both APIs show a wide spread between minimum and maximum latency at high load, indicating that individual request latency becomes less predictable under heavy load for both architectures, with GraphQL showing the higher worst-case value.

API	**50 load**	**200 load**	**500 load**
REST	2053.99 ms	2079.51 ms	2803.4 ms
GraphQL	2066.42 ms	2102.55 ms	3050.31 ms
REST min/max latency values	min: 2017.42 ms max: 2141.08 ms	min: 2017.87 ms max: 2487.53 ms	min: 2018.83 ms max: 5336.36 ms
GraphQL min/max latency values	min: 2027.68 ms max: 2178.82 ms	min: 2031.39 ms max: 2453.95 ms	min: 2034.06 ms max: 6103.99 ms

Table 5.1: Average response time (ms), with max and min values across load levels.

Throughput Analysis: The values in Table 5.2 are averaged across all query complexity levels. REST outperformed GraphQL at every load level, although the two were nearly matched at low and medium workloads. At 500 concurrent users, REST reached 103.95 req/s against GraphQL 97.16 req/s, a gap of roughly 6.5%.

API	**50 load**	**200 load**	**500 load**
REST	23.24 req/s	79.54 req/s	103.95 req/s
GraphQL	23.09 req/s	78.69 req/s	97.16 req/s

Table 5.2: Average throughput (requests/s) across load levels.

Impact of Query Complexity on Latency: Table 5.3 and Figure 5.1 break latency down by both query complexity and load. At simple and medium complexity, REST and GraphQL are closely matched across all load levels. At 500 load, GraphQL simple and medium query latency is about 2.9% and 3.3% higher than REST. The gap is far more pronounced for complex queries at 500 load, where GraphQL latency is about 14.3% higher than REST, the largest relative difference observed at any complexity level. The aggregate averages in Table

5.1 summarize all complexity levels together and therefore do not reflect these per-level differences.

Complexity	API type	50 load	200 load	500 load
Simple	REST	2047.05 ms	2070.37 ms	2436.51 ms
Simple	GraphQL	2058.98 ms	2077.59 ms	2509.9 ms
Medium	REST	2048.26 ms	2075.76 ms	2474.94 ms
Medium	GraphQL	2059.85 ms	2079.02 ms	2558.89 ms
Complex	REST	2066.66 ms	2092.40 ms	3498.74 ms
Complex	GraphQL	2080.42 ms	2151.02ms	4082.15 ms

Table 5.3: Average response time (ms) by query complexity and load.

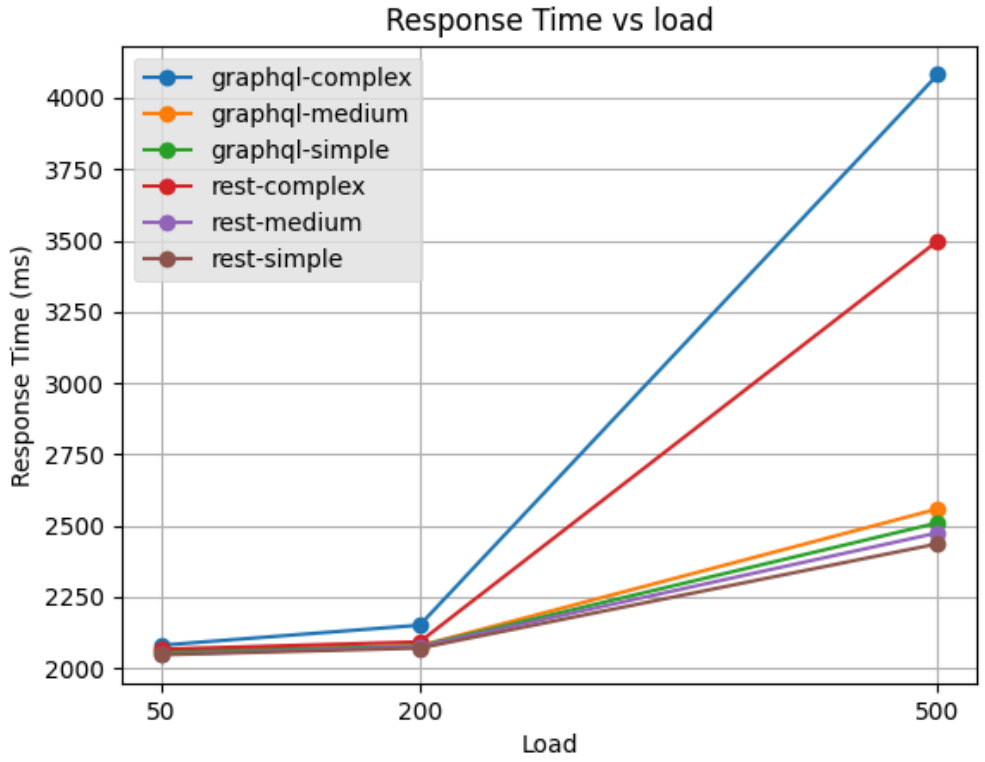


Figure 5.1: Latency trends across load (users) levels.

Impact of Query Complexity on Throughput: Table 5.4 and Figure 5.2 present the corresponding throughput breakdown. At simple and medium complexity, REST and GraphQL again perform comparably at every load level. At 500 load, GraphQL simple and medium query throughput is within roughly 3–4% of REST. At complex queries, GraphQL falls further behind, a gap of about 15%. As with latency, the gap between the two APIs is small for simple and medium requests and becomes substantially larger for complex, deeply nested requests. A detailed breakdown of total processed requests is provided in Appendix 6.

Complexity	API type	50 load	200 load	500 load
Simple	REST	23.34 req/s	79.90 req/s	116.54 req/s
Simple	GraphQL	23.17 req/s	79.58 req/s	112.94 req/s
Medium	REST	23.31 req/s	79.62 req/s	114.86 req/s
Medium	GraphQL	23.17 req/s	79.61 req/s	110.10 req/s
Complex	REST	23.08 req/s	79.01 req/s	80.46 req/s
Complex	GraphQL	22.93 req/s	76.89 req/s	68.43 req/s

Table 5.4: Average throughput (requests/s) by query complexity and load.

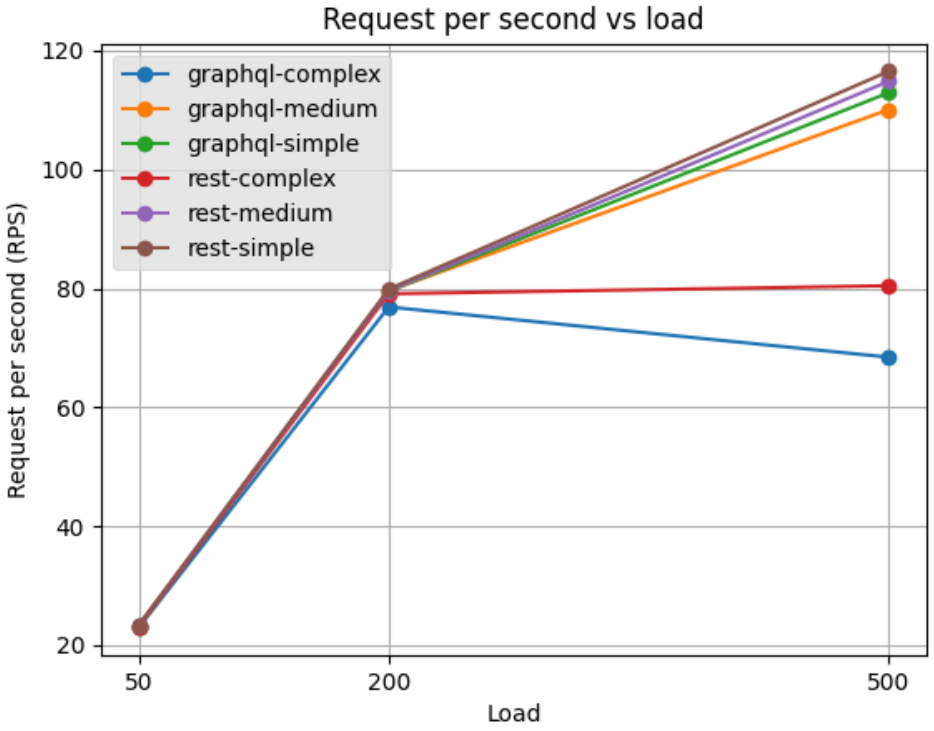


Figure 5.2: Throughput (req/s) trends across load levels.

Complexity and Performance Correlation: To evaluate performance stability, percentage changes were calculated between consecutive complexity levels (simple to medium and medium to complex) under all load scenarios, using (new value - old value) / old value × 100. Values for the 50- and 200-load scenarios are provided in Appendix 7. Both APIs changed only marginally from simple to medium complexity in latency and throughput, but diverged sharply from medium to complex queries. REST latency increased by 41.37% and throughput fell by 29.95%, while GraphQL latency increased by 59.53% and throughput fell by 37.85%. This indicates that GraphQL latency and throughput both degrade proportionally more than REST once a query requires a second level of nested relational data. Table 5.5 and Figures 5.3 and 5.4 confirm this pattern, showing throughput and latency closely matched between the two APIs at low complexity but clearly diverging at the complex tier, with GraphQL showing

both lower throughput and greater latency spread than REST under deeply nested queries at high load.

Metric	API Type	Simple -> Medium (%)	Medium -> Complex (%)
Latency Increase	REST	+1.58%	+41.37%
Latency Increase	GraphQL	+1.95%	+59.53%
Throughput Change	REST	-1.44%	-29.95%
Throughput Change	GraphQL	-2.51%	-37.85%

Table 5.5: Performance deviation across query complexity levels under high load (500 users)

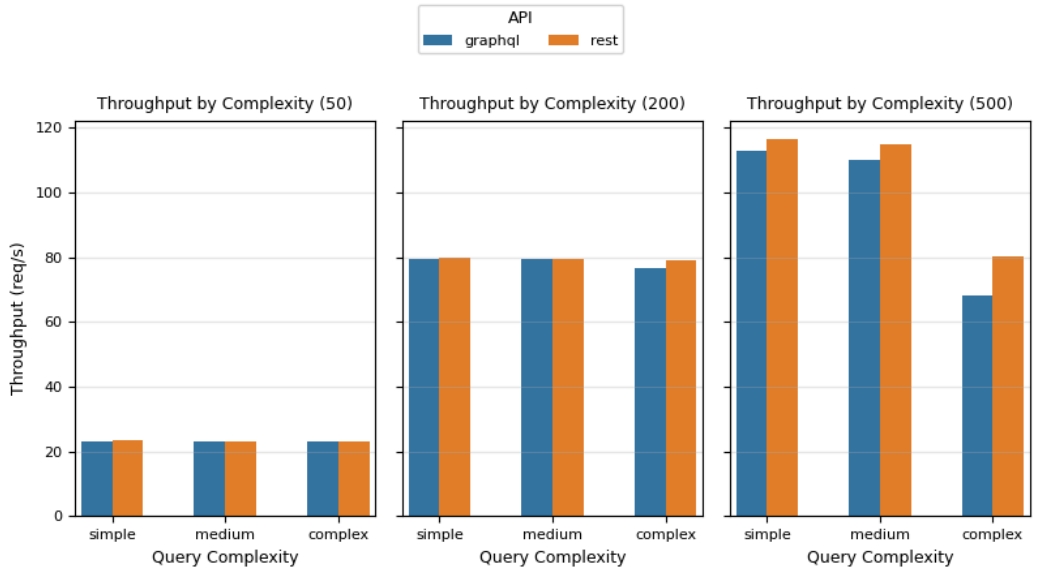


Figure 5.3: Throughput stability comparison across query complexity levels and loads.

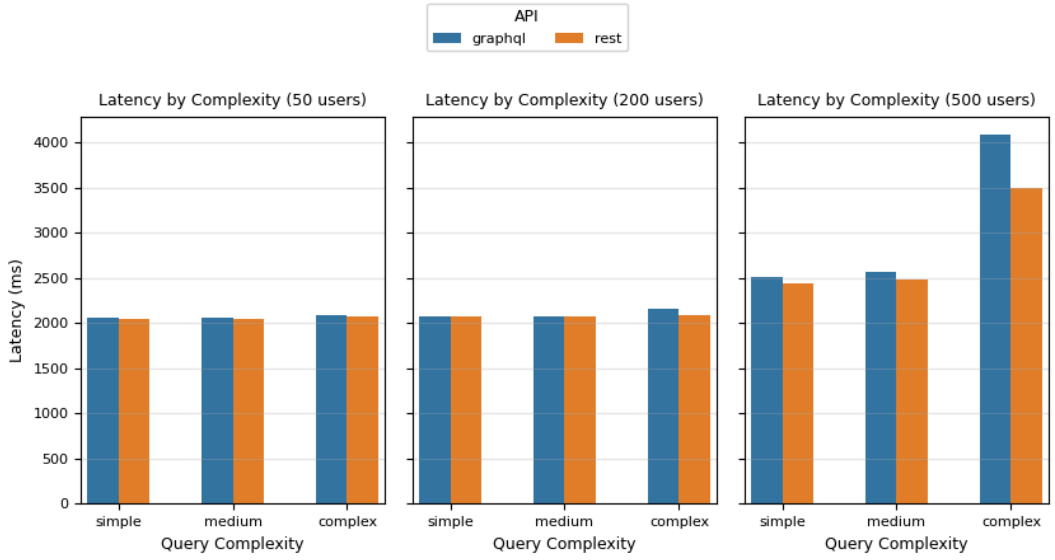


Figure 5.4: Latency stability comparison across query complexity levels and loads.

6 Analysis and Discussion

6.1 REST vs GraphQL (Latency & Throughput)

The experimental results show that latency increases for both REST and GraphQL as the number of concurrent users grows, and that the gap between the two APIs widens as workload increases. At 50 and 200 concurrent users, the gap remains small (around 1%), but widens clearly at 500 users, where REST holds an advantage of roughly 8% in latency and 6.5% in throughput. Both APIs also become less predictable at high load, where GraphQL worst-case latency (6103.99 ms) exceeds REST (5336.36 ms) by a wider margin than the averages suggest, indicating that GraphQL reaches its performance limits earlier under high concurrency.

These results are directionally consistent with the literature reporting that REST achieves lower latency and higher throughput than GraphQL under heavy load. This can be explained by REST stateless, resource-oriented architecture, which introduces relatively low server-side overhead. In contrast, GraphQL relies on query parsing, schema validation, and resolver execution increases runtime cost and contributes to earlier throughput saturation [21].

Previous research studies have reported throughput advantages of 18–37% and latency advantages of 13–51% for REST [1, 21, 22], compared with the 6.5–8% difference observed in this experiment. Variations in experimental methodology may explain this discrepancy and may have contributed to the larger performance gaps reported in previous studies. Nevertheless, Diyasa's evaluation of CRUD operations [3] reports percentage differences close to those observed in this experiment, providing additional support for the finding that REST can achieve faster response times for simple requests, while GraphQL tends to exhibit comparatively higher latency.

GraphQL may be the more suitable choice when resource utilization is a primary concern, as REST requires a greater number of client–server calls to retrieve related data, increasing the computational resources needed for processing and memory [1, 22, 23]. In a real-world evaluation of management systems, Lawi [1] found that GraphQL was 37% more efficient in CPU utilization and used 40% less memory than REST. Similarly, Chandra and Farisi [10], in their stress-testing evaluation, reported that REST exhibited higher average CPU and memory utilization than GraphQL. These findings suggest that although REST may provide advantages in latency and throughput under the conditions examined in this study, GraphQL can offer benefits in terms of resource utilization.

6.2 Query Complexity Impact

The experimental results show that increasing query complexity leads to higher latency and lower throughput for both REST and GraphQL. At 50 and 200 concurrent users, both APIs perform almost identically for simple and medium queries, with less than a 1% difference in both latency and throughput. For complex queries at 200 users, the performance gap widens

slightly to 2.7%. At 500 users, the difference becomes more noticeable, reaching 3-4% for simple and medium queries. The largest difference occurs under the highest workload (500 users with complex queries), where REST achieves 14.3% lower latency and 15% higher throughput compared to GraphQL. This represents the largest relative performance difference observed across all complexity levels. These results indicate that GraphQL latency and throughput degrade proportionally more than REST as query complexity increases. This confirms that GraphQL added overhead from query parsing and resolver execution, particularly when queries involve a second level of nested relational data.

These results consistently indicate better performance for REST when comparing the execution of a single client request in isolation. However, this comparison captures only single-request performance. In practice, retrieving related resources through REST for a comparable complex query typically requires multiple sequential requests, and the cumulative latency and network overhead associated with two or more REST round-trips may exceed that of a single equivalent GraphQL query, despite GraphQL higher per-request server-side cost.

GraphQL's advantage therefore lies in aggregating related data into a single request, reducing network round-trips and avoiding over-fetching and under-fetching [1, 4, 5, 10, 12, 29]. This benefit is not captured by single-request latency comparisons but becomes particularly relevant in complex, multi-entity retrieval scenarios, where an equivalent REST implementation may require multiple requests [5, 12, 29]. Furthermore, GraphQL can reduce over-fetching and generally transmits smaller payloads for complex queries, potentially reducing the memory required to process responses. REST is therefore the more efficient choice for isolated, single-resource requests, whereas GraphQL's relative advantage increases with the number of related resources a client needs to retrieve within a single round trip [23].

6.3 When to use REST vs GraphQL

The combined findings from the experimental evaluation and the literature indicate that selecting between REST and GraphQL requires consideration of several application-specific factors.

REST is particularly well suited to high-volume, performance-critical systems with predictable data access patterns. Its low overhead and stateless design allow it to scale efficiently under heavy load, making it appropriate for transactional systems such as banking, payment processing, and logistics platforms. REST is also advantageous for bulk data transfer and large-scale distributed systems, where HTTP caching and CDN support can considerably improve performance and reduce latency [9, 21]. However, REST becomes less efficient as query complexity increases. In scenarios involving multiple related resources often require several requests and larger payloads due to fixed response structures, leading to over-fetching and performance degradation [5, 12].

While GraphQL is generally slower in terms of raw throughput, it is better suited to applications with complex and evolving data requirements. Its ability to retrieve precisely the

required data in a single request reduces over-fetching and under-fetching, which is particularly effective for systems involving deeply nested relationships [29], such as mobile applications, dashboards, and microservices aggregation layers [27, 31], especially valuable in environments with limited bandwidth or poor network conditions, where minimizing request count and payload size is critical [4, 9]. GraphQL flexibility further supports rapid development and changing client requirements without backend modifications or versioning [21, 28]. At the same time, it introduces higher server-side overhead and reaches its performance limits earlier under high concurrency due to query parsing and resolver execution [21, 23]. It also presents challenges around caching and security, since traditional HTTP caching is less effective and query complexity must be actively controlled to avoid performance degradation or denial-of-service risk [21, 24].

Prior studies conclude that REST provides superior performance for individual requests, while others demonstrate that GraphQL performs better in complete data-retrieval scenarios involving multiple entities [5, 11, 22]. Several studies consequently recommend a hybrid approach, where REST is used for core transactional services, and GraphQL serves as an aggregation layer for client-facing applications, analytics, or dashboards that require high flexibility [9, 21].

7 Conclusions and Future Work

This thesis investigated the performance differences between REST and GraphQL with respect to latency, throughput, and query complexity. REST achieved lower latency and higher throughput than GraphQL across all tested workload and complexity levels, with the performance gap widening as both concurrency and query complexity increased. The difference was most pronounced for complex, deeply nested queries under high load.

REST continued to scale efficiently as load increased, making it well suited to high-volume, performance-critical systems with predictable data-access patterns. However, when a client must call multiple endpoints to retrieve related data, REST shifts its complexity cost to the client in the form of extra network round-trips, which can increase overall latency even when each individual server request is fast. GraphQL, in contrast, reached its performance limits earlier because of the added cost of query parsing and resolver execution, shifting its complexity cost to the server (higher latency, lower throughput per request) in exchange for letting the client retrieve all nested data in a single request, avoiding multiple round-trips.

Neither architecture is universally superior. As discussed in Section 6.3, REST suits transactional and enterprise services where scalability, low latency, and high throughput are critical, while GraphQL suits applications needing flexible, complex data retrieval, such as mobile clients and dashboards. A hybrid approach, using REST for public APIs and simple resources, and GraphQL for dashboards or mobile clients, may offer the most practical balance for systems that need both. Ultimately, the choice should be grounded in the specific conditions of the system being built, including the nature of related-data relationships, server-side processing cost, and network-level efficiency.

This study is limited to the specific experimental setup evaluated, a single local SQLite-backed environment, three fixed levels of query complexity, and a maximum of 500 concurrent users. CPU and memory utilization, caching strategies, authentication, and network-level conditions such as bandwidth constraints were outside its scope. Future work could extend this study by measuring CPU, memory, and network utilization directly under the same complexity and workload design used here, by repeating the experiment over a real network connection rather than a local backend, to isolate the round-trip and payload-size advantages the literature attributes to GraphQL, and by evaluating hybrid architectures that combine REST for performance-critical operations with GraphQL as an aggregation layer, to test whether such a combination captures the benefits of both approaches identified in this and prior work.

References

[1] Lawi, Armin, Benny LE Panggabean, and Takaichi Yoshida. "Evaluating graphql and rest api services performance in a massive and intensive accessible information system." Computers 10.11 (2021): 138. Available: https://www.mdpi.com/2073-431X/10/11/138 (accessed 2026-02-02)

[2] Sayago Heredia, Jaime, Evelin Flores-García, and Andres Recalde Solano. "Comparative analysis between standards oriented to web services: SOAP, REST and GRAPHQL." International conference on applied technologies. Cham: Springer International Publishing, 2019. Available: https://link.springer.com/chapter/10.1007/978-3-030-42517-3_22 (accessed 2026-02-02)

[3] Diyasa, GS Mas, et al. "Comparative analysis of rest and graphql technology on nodejs-based api development." Nusantara Science and Technology Proceedings (2021): 43-52. Available: https://pdfs.semanticscholar.org/0d4d/5d7481e71907955e7bbb259cf5b2bd1ea8de.pdf (accessed 2026-02-02)

[4] Khan, Irfan Ahmed, Harsh Mishra, and Khushboo Choubey. "A Comparative Analysis of REST and GraphQL APIs: Performance, Efficiency, and Developer Experience." International Journal of Advanced Multidisciplinary Scientific Research 8.4 (2025): 29-39. Available: https://www.ijamsr.com/issues/6_Volume%208_Issue%204/20250412_070814_8212.pdf (accessed 2026-02-02)

[5] Vadlamani, Sri Lakshmi, et al. "Can graphql replace rest? a study of their efficiency and viability." 2021 IEEE/ACM 8th International Workshop on Software Engineering Research and Industrial Practice (SER&IP). IEEE, 2021. Available: https://ieeexplore.ieee.org/abstract/document/9474834 (accessed 2026-02-02)

[6] Brito, Gleison, and Marco Tulio Valente. "REST vs GraphQL: A controlled experiment." 2020 IEEE international conference on software architecture (ICSA). IEEE, 2020. Available**:** https://ieeexplore.ieee.org/abstract/document/9101226 (accessed 2026-02-04)

[7] Jin, Runjie, et al. "GraphQL vs. REST: Investigating Performance and Scalability for Serverless Data Persistence." 2025 IEEE International Conference on Cloud Engineering (IC2E). IEEE, 2025. Available: https://ieeexplore.ieee.org/abstract/document/11196547 (accessed 2026-03-11)

[8] Mikuła, Mateusz, and Mariusz Dzieńkowski. "Comparison of REST and GraphQL web technology performance." *Journal of Computer Sciences Institute* 16 (2020): 309-316. Available: https://ph.pollub.pl/index.php/jcsi/article/view/2077 (accessed 2026-03-11)

[9] Jangala, Vinod Kumar. "Comparative analysis of REST and GraphQL APIs in large-scale enterprise applications." International Journal of Contemporary Research in Multidisciplinary 2.1 (2023): 94-102. Available: https://multiarticlesjournal.com/uploads/articles/IJCRM-2023-2-1-20.pdf (accessed 2026-03-11)

[10] Chandra, Steven, and Ahmad Farisi. "Comparative Analysis of RESTful, GraphQL, and gRPC APIs: Performance Insight from Load and Stress Testing." Jurnal Sisfokom (Sistem Informasi dan Komputer) 14.1 (2025): 81-85. Available: https://jurnal.atmaluhur.ac.id/index.php/sisfokom/article/view/2315/1300 (accessed 2026-03-11)

[11] Gluhak, Marko, and Marjan Hericko. "Comparison of Graph and REST APIs." SQAMIA. 2023. Available: https://ceur-ws.org/Vol-3588/p4.pdf (accessed 2026-03-12)

[12] Fhonna, Rizky Putra, et al. "Performance analysis of API protocol models as recommendations for developers in application development." JINAV: Journal of Information and Visualization 5.2

(2024): 272-279. Available: https://journal.mattawangmediatama.co.id/index.php/jinav/article/view/3041 (accessed 2026-03-12)

[13] Wieringa, Roel. "Design science as nested problem solving." Proceedings of the 4th international conference on design science research in information systems and technology. 2009. Available: https://dl.acm.org/doi/abs/10.1145/1555619.1555630 (accessed 2026-03-12)

[14] Hevner, Alan R., et al. "Design science in information systems research1." MIS quarterly 28.1 (2004): 75-106. Available: https://misq.umn.edu/misq/article-abstract/28/1/75/261/Design-Science-in-Information-Systems-Research1?redirectedFrom=fulltext (accessed 2026-03-12)

[15] RESTful API Tutorial, "What is REST API?", Available: https://restfulapi.net/ (accessed 2026-03-18)

[16] GraphQL Foundation, "Introduction to GraphQL", Available: https://graphql.org/learn/ (accessed 2026-03-18).

[17] The Python Tutorial, Available: https://docs.python.org/3/tutorial/index.html (accessed 2026-04-06).

[18] Flask, "User's Guide", Available: https://flask.palletsprojects.com/en/stable/ (accessed 2026-04-06).

[19] SQLite, "Appropriate Uses For SQLite", Available: https://sqlite.org/whentouse.html (accessed 2026-04-06).

[20] Locust, "What is Locust?", Available: https://docs.locust.io/en/stable/what-is-locust.html (accessed 2026-04-06).

[21] Devalla, Sireesha. "EVALUATING THE TRANSITION TO WEBFLUX PERFORMANCE, MAINTAINABILITY, AND ADOPTION CHALLENGES IN MODERN JAVA WEB FRAMEWORKS." Journal ID 2811 (2022): 6201. Available: https://www.researchgate.net/profile/Sireesha-Devalla/publication/397557136_Enterprise-Scale_Evaluation_of_REST_and_GraphQL_Balancing_Performance_Scalability_and_Resource_Utilization/links/69154b01c5e6981fc477f6b2/Enterprise-Scale-Evaluation-of-REST-and-GraphQL-Balancing-Performance-Scalability-and-Resource-Utilization.pdf (accessed 2026-04-09).

[22] Goriparthi, Sudeesh. "Streamlining API development: a comparative analysis of GraphQL and RESTful web services." INTERNATIONAL JOURNAL OF DATA ANALYTICS RESEARCH AND DEVELOPMENT (IJDARD) 2 (2023): 59-71. Available: https://openlibindex.com/index.php/IJDARD/article/view/IJDARD_02_01_007 (accessed 2026-04-09).

[23] González, Gregorio Sebastián Gualavisí, et al. "Development of REST and GraphQL APIs: Application Programming." Journal of Software Engineering and Computing 1.1 (2026): 26-26. Available: https://journal.kernelxos.com/jsec/article/view/7 (accessed 2026-04-09).

[24] Cha, Alan, et al. "A principled approach to GraphQL query cost analysis." Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the

Foundations of Software Engineering. 2020. Available: https://doi.org/10.1145/3368089.3409670 (accessed 2026-04-18).

[25] IEEE Xplore (Institute of Electrical and Electronics Engineers), “About IEEE *Xplore*” Available: https://ieeexplore.ieee.org/Xplorehelp/overview-of-ieee-xplore/about-ieee-xplore (accessed 2026-04-18)

[26] Google Scholar, “About”. Available: https://scholar.google.com/intl/en/scholar/about.html (accessed 2026-04-18)

[27] Ambasht, Anshumali. "API Integration using GraphQL." *International Journal of Computer Trends and Technology* 71.8 (2023): 28-33. Available: https://www.researchgate.net/profile/Anshumali-Ambasht/publication/374327841_API_Integration_using_GraphQL/links/6521a0f7fc5c2a0c3bbff15f/API-Integration-using-GraphQL.pdf (accessed 2026-04-18)

[28] Papadhopulli, Ina, and Hakik Paci. "GraphQL: A comprehensive analysis of its advantages, challenges, and best practices in modern API development." The Eurasia Proceedings of Science Technology Engineering and Mathematics 32 (2024): 230-237. Available: https://dergipark.org.tr/en/pub/epstem/article/1598445 (accessed 2026-04-18)

[29] Veeravalli, S. D. "Next-Generation APIs for CRM: A Study on GraphQL Implementation for Salesforce Data Integration." Journal ID 4951 (2023): 3268. Available: https://www.researchgate.net/profile/Shivakrishna-Veeravalli-3/publication/390933123_Next-Generation_APIs_for_CRM_A_Study_on_GraphQL_Implementation_for_Salesforce_Data_Integration/links/681c5f8260241d51402508a9/Next-Generation-APIs-for-CRM-A-Study-on-GraphQL-Implementation-for-Salesforce-Data-Integration.pdf (accessed 2026-04-18)

[30] Serbout, Souhaila, et al. "API Rate Limit Adoption--A pattern collection." Proceedings of the 28th European Conference on Pattern Languages of Programs. 2023. Available: https://doi.org/10.1145/3628034.3628039 (accessed 2026-04-18)

[31] Goriparthi, Sudeesh. "ENHANCING DATA QUERY EFFICIENCY WITH GRAPHQL APIS: A COMPARATIVE STUDY WITH RESTFUL WEB SERVICES." INTERNATIONAL JOURNAL OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING (IJAIML) 3: 132-143. Available: https://openlibindex.com/index.php/IJAIML/article/view/IJAIML_03_01_013 (accessed 2026-04-18)

[32] SQLAlchemy, “Overview”. Available: https://docs.sqlalchemy.org/en/20/intro.html (accessed 2026-04-22)

[33] Graphene-Python, "Introduction", Available: https://docs.graphene-python.org/en/latest/ (accessed 2026-06-03)

A Appendix

AI Statement

Two AI tools were used in this thesis: Grammarly and ChatGPT.

The primary tool was Grammarly, which was used to assist with English language corrections, including spelling, grammar, and sentence structure where necessary. Grammarly provided continuous writing assistance throughout the thesis writing process. ChatGPT was used for two primary purposes. First, it served as a programming assistant during the implementation of the experimental environment. Since the objective of this thesis was to compare the performance of two API architectures, it was important to ensure that both implementations were correct, efficient, and comparable. ChatGPT was therefore used to review code, identify implementation errors, suggest optimizations where appropriate, and assist with debugging. It functioned as a supporting tool throughout the development process. Second, ChatGPT was occasionally used to improve the language and readability of the thesis. It assisted with refining sentence structure, correcting grammar, and improving the overall flow of the text. The prompt used for this purpose was: "*Correct text formatting to improve readability. Here is my text*:"

Appendix 1:

```python

def get_db():
    if "db" not in g:
        g.db = SessionLocal()
    return g.db

@app.teardown_appcontext
def teardown_db(exception):
    db = g.pop("db", None)
    if db is not None:
        db.close()

# Simple Query
@app.route("/user/<int:user_id>")
def get_user(user_id):
    session = get_db()
    user = session.get(User, user_id)

    if not user: return jsonify({"error": "User not found"}), 404

    return jsonify({
        "id": user.id,
        "name": user.name
    })

# Medium Query
@app.route("/user/<int:user_id>/posts")
def get_posts(user_id):
    session = get_db()
    user = (
        session.query(User)
        .options(joinedload(User.posts))
        .filter(User.id == user_id)
        .first()
    )

    if not user: return jsonify({"error": "User not found"}), 404

    return jsonify([
        {"id": p.id, "title": p.title}
        for p in user.posts
    ])

# Complex Query
@app.route("/user/<int:user_id>/full")
def get_full(user_id):
    session = get_db()
    user = (
        session.query(User)
        .options(joinedload(User.posts).joinedload(Post.comments))
        .filter(User.id == user_id)
        .first()
    )

    if not user: return jsonify({"error": "User not found"}), 404

    return jsonify({
        "id": user.id,
        "posts": [
            {
                "title": p.title,
                "comments": [c.text for c in p.comments]
            } for p in user.posts
        ]
    })

```

```python
# GraphQL Types
class CommentType(graphene.ObjectType):
    text = graphene.String()

class PostType(graphene.ObjectType):
    title = graphene.String()
    comments = graphene.List(CommentType)

    def resolve_comments(parent, info):
        return parent.comments

class UserType(graphene.ObjectType):
    id = graphene.Int()
    name = graphene.String()
    posts = graphene.List(PostType)

    def resolve_posts(parent, info):
        return parent.posts


# --- Helpers for the resolver ---
def _get_field_nodes(info):
    """ Return the list of field nodes for the incoming GraphQL query. """
    return getattr(info, "field_nodes", None) or getattr(info, "field_asts")

def _selected_field_names(field_node):
    """Return the set of field names selected directly under a field node."""
    if not field_node.selection_set: return set()
    return {
        sel.name.value
        for sel in field_node.selection_set.selections
            if hasattr(sel, "name")
    }

def _find_field_node(selections, name):
    """Find a specific field node by name among a list of selections."""
    for sel in selections:
        if hasattr(sel, "name") and sel.name.value == name:
            return sel
    return None

# Query
class Query(graphene.ObjectType):
    user = graphene.Field(UserType, id=graphene.Int())

    def resolve_user(root, info, id):
        session = info.context["session"]

        # fields (id, name, posts)
        field = _get_field_nodes(info)[0]
        top_level = _selected_field_names(field)

        query = session.query(User)

        if "posts" in top_level:
            posts_node = _find_field_node(field.selection_set.selections, "posts")
            nested = _selected_field_names(posts_node) if posts_node else set()
```

```python
# Query
class Query(graphene.ObjectType):
    user = graphene.Field(UserType, id=graphene.Int())

    def resolve_user(root, info, id):
        session = info.context["session"]

        # fields (id, name, posts)
        field = _get_field_nodes(info)[0]
        top_level = _selected_field_names(field)

        query = session.query(User)

        if "posts" in top_level:
            posts_node = _find_field_node(field.selection_set.selections, "posts")
            nested = _selected_field_names(posts_node) if posts_node else set()

            if "comments" in nested:
                # complex query: needs posts AND comments => two joins, as REST /user/<user_id>/full
                query = query.options(
                    joinedload(User.posts).joinedload(Post.comments)
                )
            else:
                # medium query: needs posts only => one join, as REST /user/<user_id>/posts
                query = query.options(joinedload(User.posts))

        # else: simple query (id, name only) => no join at all, as REST's /user/<user_id>
        return query.filter(User.id == id).first()

schema = graphene.Schema(query=Query)

# Flask App
app = Flask(__name__)

@app.route("/graphql", methods=["POST"])
def graphql_server():
    session = SessionLocal()

    try:
        data = request.get_json()
        result = schema.execute(
            data.get("query"),
            context_value={"session": session}
        )

        # Handle errors
        if result.errors:
            return jsonify({
                "errors": [str(e) for e in result.errors]
            }), 500

        return jsonify(result.data)
    finally:
        session.close()


if __name__ == "__main__":
    app.run(port=5001, debug=True)
```

Appendix 2:

```python
from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship, declarative_base

Base = declarative_base()

class User(Base):
    """ User model """
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    name = Column(String)
    posts = relationship("Post", back_populates="user")

class Post(Base):
    """ Post model """
    __tablename__ = "posts"
    id = Column(Integer, primary_key=True)
    title = Column(String)
    user_id = Column(Integer, ForeignKey("users.id"))
    user = relationship("User", back_populates="posts")
    comments = relationship("Comment", back_populates="post")

class Comment(Base):
    """ Comment model """
    __tablename__ = "comments"
    id = Column(Integer, primary_key=True)
    text = Column(String)
    post_id = Column(Integer, ForeignKey("posts.id"))
    post = relationship("Post", back_populates="comments")

```

```python
from db import engine, SessionLocal
from models import Base, User, Post, Comment

# Reset the database
Base.metadata.drop_all(engine)
Base.metadata.create_all(engine)

session = SessionLocal()

NUM_USERS = 500
POSTS_PER_USER = 10
COMMENTS_PER_POST = 10

for i in range(NUM_USERS):
    user = User(name=f"User {i}")

    for j in range(POSTS_PER_USER):
        post = Post(title=f"User {i} - Post {j}")

        for k in range(COMMENTS_PER_POST):
            comment = Comment(text=f"Comment {k}")
            post.comments.append(comment)

        user.posts.append(post)
    session.add(user)

session.commit()
session.close()

print("Database reset and seeded!")
```

Appendix 3:

```
graphql_complex_200_run1.csv  graphql_medium_50_run4.csv   rest_complex_200_run2.csv  rest_medium_50_run5.csv
graphql_complex_200_run2.csv  graphql_medium_50_run5.csv   rest_complex_200_run3.csv  rest_medium_500_run1.csv
graphql_complex_200_run3.csv  graphql_medium_500_run1.csv  rest_complex_200_run4.csv  rest_medium_500_run2.csv
graphql_complex_200_run4.csv  graphql_medium_500_run2.csv  rest_complex_200_run5.csv  rest_medium_500_run3.csv
graphql_complex_200_run5.csv  graphql_medium_500_run3.csv  rest_complex_50_run1.csv   rest_medium_500_run4.csv
graphql_complex_50_run1.csv   graphql_medium_500_run4.csv  rest_complex_50_run2.csv   rest_medium_500_run5.csv
graphql_complex_50_run2.csv   graphql_medium_500_run5.csv  rest_complex_50_run3.csv   rest_simple_200_run1.csv
graphql_complex_50_run3.csv   graphql_simple_200_run1.csv  rest_complex_50_run4.csv   rest_simple_200_run2.csv
graphql_complex_50_run4.csv   graphql_simple_200_run2.csv  rest_complex_50_run5.csv   rest_simple_200_run3.csv
graphql_complex_50_run5.csv   graphql_simple_200_run3.csv  rest_complex_500_run1.csv  rest_simple_200_run4.csv
graphql_complex_500_run1.csv  graphql_simple_200_run4.csv  rest_complex_500_run2.csv  rest_simple_200_run5.csv
graphql_complex_500_run2.csv  graphql_simple_200_run5.csv  rest_complex_500_run3.csv  rest_simple_50_run1.csv
graphql_complex_500_run3.csv  graphql_simple_50_run1.csv   rest_complex_500_run4.csv  rest_simple_50_run2.csv
graphql_complex_500_run4.csv  graphql_simple_50_run2.csv   rest_complex_500_run5.csv  rest_simple_50_run3.csv
graphql_complex_500_run5.csv  graphql_simple_50_run3.csv   rest_medium_200_run1.csv   rest_simple_50_run4.csv
graphql_medium_200_run1.csv   graphql_simple_50_run4.csv   rest_medium_200_run2.csv   rest_simple_50_run5.csv
graphql_medium_200_run2.csv   graphql_simple_50_run5.csv   rest_medium_200_run3.csv   rest_simple_500_run1.csv
graphql_medium_200_run3.csv   graphql_simple_500_run1.csv  rest_medium_200_run4.csv   rest_simple_500_run2.csv
graphql_medium_200_run4.csv   graphql_simple_500_run2.csv  rest_medium_200_run5.csv   rest_simple_500_run3.csv
graphql_medium_200_run5.csv   graphql_simple_500_run3.csv  rest_medium_50_run1.csv    rest_simple_500_run4.csv
graphql_medium_50_run1.csv    graphql_simple_500_run4.csv  rest_medium_50_run2.csv    rest_simple_500_run5.csv
graphql_medium_50_run2.csv    graphql_simple_500_run5.csv  rest_medium_50_run3.csv
graphql_medium_50_run3.csv    rest_complex_200_run1.csv    rest_medium_50_run4.csv
```



```python
import random

from locust import HttpUser, task, between

class RestAPIUser(HttpUser):
    host = "http://localhost:5000"

    # Get a random user to query (1-500)
    def get_random_user(self):
        return random.randint(1, 500)

    @task(1)
    def simple_query(self):
        user_id = self.get_random_user()
        self.client.get(f"/user/{user_id}")

    @task(1)
    def medium_query(self):
        user_id = self.get_random_user()
        self.client.get(f"/user/{user_id}/posts")

    @task(1)
    def complex_query(self):
        user_id = self.get_random_user()
        self.client.get(f"/user/{user_id}/full")

```

```python

# Graphql USER
class GraphQLAPIUser(HttpUser):
    host = "http://localhost:5001"

    def get_random_user(self):
        return random.randint(1, 500)

    @task(1)
    def simple_query(self):
        user_id = self.get_random_user()
        query = f"""
        {{
            user(id: {user_id}) {{
                id
                name
            }}
        }}
        """
        self.client.post("/graphql", json={"query": query})

    @task(1)
    def medium_query(self):
        user_id = self.get_random_user()
        query = f"""
        {{
            user(id: {user_id}) {{
                posts {{
                    title
                }}
            }}
        }}
        """
        self.client.post("/graphql", json={"query": query})

    @task(1)
    def complex_query(self):
        user_id = self.get_random_user()
        query = f"""
        {{
            user(id: {user_id}) {{
                posts {{
                    comments {{
                        text
                    }}
                }}
            }}
        }}
        """
        self.client.post("/graphql", json={"query": query})

```

Appendix 4:

```python
# path= ../results/*.csv
def load_data(path="results/*.csv") -> pd.DataFrame:
    """Load data from CSV files"""
    files = glob.glob(path)
    #print("FILES FOUND:", files)
    data = []

    for file in files:
        df = pd.read_csv(file)
        df.columns = df.columns.str.strip()

        # Extract aggregated row
        agg_rows = df[df["Name"] == "Aggregated"]
        if agg_rows.empty:
            print(f"No Aggregated row in {file}")
            continue

        agg = agg_rows.iloc[0]

        # Extract metadata from filename
        name = file.split("/")[-1].replace(".csv", "")
        parts = name.split("_")

        api = parts[0]
        complexity = parts[1]
        load = int(parts[2])

        data.append({
            "api": api,
            "complexity": complexity,
            "load": load,
            "Average latency": agg["Average Response Time"],
            "max latency": agg["Max Response Time"],
            "min latency": agg["Min Response Time"],
            "throughput": agg["Requests/s"],
            "requests": agg["Request Count"],
        })

    df_final = pd.DataFrame(data)
    return df_final


def aggregate_data(df: pd.DataFrame, print_results=True):
    """ Aggregate data from multiple files."""
    if df.empty:
        print("ERROR: No data collected.")
        return None

    # Extract API name from path
    df["api"] = df["api"].apply(lambda x: os.path.basename(str(x)))

    df_grouped = df.groupby(
        ["api", "complexity", "load"],
    ).mean(numeric_only=True).reset_index()

    if print_results:
        print("\nFinal Results:")
        print(df_grouped.to_string(index=False))

    return df_grouped


def print_averages_by_load(df_grouped: pd.DataFrame) -> pd.DataFrame:
    """Print average latency and throughput across complexities for each API/load."""
    metric_columns = [
        "Average latency",
        "max latency",
```

```
        "Average latency",
        "max latency",
        "min latency",
        "throughput",
    ]
    summary_rows = []

    for api in sorted(df_grouped["api"].unique()):
        api_data = df_grouped[df_grouped["api"] == api]

        for load in sorted(api_data["load"].unique()):
            load_data = api_data[api_data["load"] == load]

            summary_row = {
                "api": api,
                "load": load,
            }

            for column in metric_columns:
                summary_row[f"{column} avr"] = (
                    load_data[column].astype(float).mean()
                )

            summary_rows.append(summary_row)

    summary = pd.DataFrame(summary_rows)
    summary = summary.round(2)

    print("\nAverage latency and throughput summary:\n")
    print(summary.to_string(index=False))

    return summary


def print_performance_deviation_by_complexity(df_grouped: pd.DataFrame, load_level=500) -> pd.DataFrame:
    """Print percentage changes between complexity levels for one load level."""
    metric_columns = [
        "Average latency",
        "throughput",
    ]
    complexity_pairs = [
        ("simple", "medium"),
        ("medium", "complex"),
    ]
    high_load = df_grouped[df_grouped["load"] == load_level]
    deviation_rows = []

    for api in sorted(high_load["api"].unique()):
        api_data = high_load[high_load["api"] == api].set_index("complexity")

        for old_complexity, new_complexity in complexity_pairs:
            if old_complexity not in api_data.index or new_complexity not in api_data.index:
                continue

            deviation_row = {
                "api": api,
                "load": load_level,
                "change": f"{old_complexity} to {new_complexity}",
            }

            for column in metric_columns:
                old_value = float(api_data.loc[old_complexity, column])
                new_value = float(api_data.loc[new_complexity, column])

                if old_value == 0:
                    percentage_change = None
```

```
                if old_value == 0:
                    percentage_change = None
                else:
                    percentage_change = ((new_value - old_value) / old_value) * 100

                deviation_row[f"{column} change (%)"] = percentage_change

            deviation_rows.append(deviation_row)

    deviation = pd.DataFrame(deviation_rows).round(2)

    print(f"\nPerformance deviation across query complexity levels ({load_level} users):\n")
    print(deviation.to_string(index=False))

    return deviation


def plot_latency_vs_load(df_grouped):
    """ Plot latency vs workload (users) for each API. RQ1."""
    plt.figure()

    for api in df_grouped["api"].unique():
        subset = df_grouped[df_grouped["api"] == api]

        for complexity in subset["complexity"].unique():
            data_plot = subset[subset["complexity"] == complexity]

            plt.plot(
                data_plot["load"],
                data_plot["Average latency"],
                marker='o',
                label=f"{api}-{complexity}"
            )

    plt.xlabel("Load")
    plt.ylabel("Response Time (ms)")
    plt.title("Response Time vs load")
    plt.xticks(sorted(df_grouped["load"].unique()))
    plt.style.use("ggplot")
    plt.legend()
    plt.grid()
    plt.show()


def plot_throughput_vs_load(df_grouped):
    """ Plot throughput vs workload for each API. RQ1."""
    plt.figure()

    for api in df_grouped["api"].unique():
        subset = df_grouped[df_grouped["api"] == api]

        for complexity in subset["complexity"].unique():
            data_plot = subset[subset["complexity"] == complexity]

            plt.plot(
                data_plot["load"],
                data_plot["throughput"],
                marker='o',
                label=f"{api}-{complexity}"
            )

    plt.xlabel("Load")
    plt.ylabel("Request per second (RPS)")
    plt.title("Request per second vs load")
```

```
    plt.title("Request per second vs load")
    plt.xticks(sorted(df_grouped["load"].unique()))
    plt.style.use("ggplot")
    plt.legend()
    plt.grid()
    plt.show()


def plot_throughput_vs_complexity(df_grouped):
    """ Plot throughput vs complexity for each API and load level. RQ2. """
    data_plot = df_grouped.copy()
    data_plot["complexity"] = pd.Categorical(
        data_plot["complexity"],
        categories=["simple", "medium", "complex"],
        ordered=True,
    )
    data_plot = data_plot.sort_values(["load", "complexity", "api"])
    load_levels = sorted(data_plot["load"].unique())

    fig, axes = plt.subplots(1, len(load_levels), figsize=(8, 3.2), sharey=True)

    for ax, load in zip(axes, load_levels):
        load_data = data_plot[data_plot["load"] == load]

        sns.barplot(
            data=load_data,
            x="complexity",
            y="throughput",
            hue="api",
            ax=ax,
            width=0.55,
        )

        ax.set_title(f"Throughput by Complexity ({load})", fontsize=9)
        ax.set_xlabel("Query Complexity", fontsize=9)
        ax.set_ylabel("Throughput (req/s)" if ax == axes[0] else "", fontsize=9)
        ax.tick_params(axis="both", labelsize=8)
        ax.margins(x=0.02)
        ax.grid(axis="y", alpha=0.3)
        ax.legend_.remove()

    handles, labels = axes[0].get_legend_handles_labels()
    fig.legend(
        handles,
        labels,
        title="API",
        loc="upper center",
        ncol=len(labels),
        fontsize=8,
        title_fontsize=9,
    )
    fig.suptitle("Throughput by Query Complexity and Load", y=1.12, fontsize=11)
    plt.tight_layout()
    plt.show()


def plot_latency_distribution(df):
    """ Plot average latency by complexity for each API and load level. RQ2 && RQ3. """
    data_plot = aggregate_data(df, print_results=False)
    data_plot["complexity"] = pd.Categorical(
        data_plot["complexity"],
        categories=["simple", "medium", "complex"],
        ordered=True,
    )
```

```
        ordered=True,
    )
    data_plot = data_plot.sort_values(["load", "complexity", "api"])
    load_levels = sorted(data_plot["load"].unique())

    fig, axes = plt.subplots(1, len(load_levels), figsize=(8, 3.2), sharey=True)

    for ax, load in zip(axes, load_levels):
        load_data = data_plot[data_plot["load"] == load]

        sns.barplot(
            data=load_data,
            x="complexity",
            y="Average latency",
            hue="api",
            ax=ax,
            width=0.55,
        )

        ax.set_title(f"Latency by Complexity ({load} users)", fontsize=9)
        ax.set_xlabel("Query Complexity", fontsize=9)
        ax.set_ylabel("Latency (ms)" if ax == axes[0] else "", fontsize=9)
        ax.tick_params(axis="both", labelsize=8)
        ax.margins(x=0.02)
        ax.grid(axis="y", alpha=0.3)
        ax.legend_.remove()

    handles, labels = axes[0].get_legend_handles_labels()
    fig.legend(
        handles,
        labels,
        title="API",
        loc="upper center",
        ncol=len(labels),
        fontsize=8,
        title_fontsize=9,
    )
    fig.suptitle("Average Latency by Query Complexity and Load", y=1.12, fontsize=11)
    plt.tight_layout()
    plt.show()


if __name__ == "__main__":
    df_raw = load_data()
    #df_grouped = aggregate_data(df_raw, print_results=False)
    df_grouped = aggregate_data(df_raw)

    if df_grouped is not None:
        print_averages_by_load(df_grouped)
        print_performance_deviation_by_complexity(df_grouped, load_level=50)
        plot_throughput_vs_load(df_grouped)
        plot_latency_vs_load(df_grouped)
        plot_throughput_vs_complexity(df_grouped)
        plot_latency_distribution(df_raw)
```

Appendix 5:

```
Final Results:
    api complexity  load  Average latency  max latency  min latency  throughput  requests
graphql    complex    50      2080.425891   2163.08310   2038.82026   22.932608    2751.8
graphql    complex   200      2151.023549   2901.84458   2045.82106   76.889580    9227.4
graphql    complex   500      4082.148025   8179.14984   2048.98946   68.432489    8226.6
graphql     medium    50      2059.852539   2124.92754   2020.96630   23.172268    2777.8
graphql     medium   200      2079.024571   2220.81812   2024.85692   79.614776    9553.4
graphql     medium   500      2558.887102   4974.20866   2025.25602  110.104361   13246.8
graphql     simple    50      2058.977623   2248.45136   2023.25852   23.175595    2779.2
graphql     simple   200      2077.590206   2239.19230   2023.47840   79.578082    9549.0
graphql     simple   500      2509.906144   5158.59740   2027.93332  112.942340   13574.8
   rest    complex    50      2066.661622   2166.10262   2028.32454   23.076617    2768.0
   rest    complex   200      2092.401505   2399.36384   2030.71358   79.097587    9493.0
   rest    complex   500      3498.737740   7064.73906   2031.78194   80.458205    9682.8
   rest     medium    50      2048.262756   2140.96986   2012.78322   23.308326    2796.6
   rest     medium   200      2075.757656   2570.54150   2012.36462   79.622526    9555.6
   rest     medium   500      2474.939368   4517.72352   2013.72212  114.865835   13793.4
   rest     simple    50      2047.049925   2116.17978   2011.13964   23.338758    2799.0
   rest     simple   200      2070.374708   2492.68614   2010.54312   79.899948    9586.2
   rest     simple   500      2436.513541   4426.60310   2010.98554  116.540826   14007.4

Average latency and throughput summary:

    api  load  Average latency avr  max latency avr  min latency avr  throughput avr
graphql    50              2066.42          2178.82          2027.68           23.09
graphql   200              2102.55          2453.95          2031.39           78.69
graphql   500              3050.31          6103.99          2034.06           97.16
   rest    50              2053.99          2141.08          2017.42           23.24
   rest   200              2079.51          2487.53          2017.87           79.54
   rest   500              2803.40          5336.36          2018.83          103.95
```

Appendix 6:

Complexity	**API type**	**50 users**	**200 users**	**500 users**
Simple	REST	2799 requests	9586 requests	14007 requests
Simple	GraphQL	2779 requests	9549 requests	13575 requests
Medium	REST	2796 requests	9555 requests	13793 requests
Medium	GraphQL	2778 requests	9553 requests	13247 requests
Complex	REST	2768 requests	9493 requests	9683 requests
Complex	GraphQL	2752 requests	9227 requests	8227 requests

Table: Total processed requests over a two-minute runtime.

Appendix 7:

Performance deviation across query complexity levels (500 users):

api	load	change	Average latency change (%)	throughput change (%)
graphql	500	simple to medium	1.95	-2.51
graphql	500	medium to complex	59.53	-37.85
rest	500	simple to medium	1.58	-1.44
rest	500	medium to complex	41.37	-29.95

Performance deviation across query complexity levels (200 users):

api	load	change	Average latency change (%)	throughput change (%)
graphql	200	simple to medium	0.07	0.05
graphql	200	medium to complex	3.46	-3.42
rest	200	simple to medium	0.26	-0.35
rest	200	medium to complex	0.80	-0.66

Performance deviation across query complexity levels (50 users):

api	load	change	Average latency change (%)	throughput change (%)
graphql	50	simple to medium	0.04	-0.01
graphql	50	medium to complex	1.00	-1.03
rest	50	simple to medium	0.06	-0.13
rest	50	medium to complex	0.90	-0.99